

1 Plan qualité

Stratégie de tests par couche, conventions, workflow Git, CI/CD et onboarding : de quoi produire des livrables fiables et intégrer un nouveau développeur rapidement.

1.1 Definition of Done

Conditions qu’une User Story doit toutes remplir pour être considérée comme terminée :

- Les critères d’acceptation (Given/When/Then) sont tous satisfaits.
- Le code est relu et approuvé par au moins un pair via une Pull Request.
- Les tests de la story sont écrits et passent (unitaires et, si concernés, intégration).
- La CI est verte : lint et tests automatiques réussis (le merge est bloqué sinon).
- La documentation utile est à jour (README du service, doc d’API, Confluence si besoin).
- Aucune régression : un bug corrigé est couvert par un test de non-régression.

RÈGLE D'OR PRODUIT

L’IA n’émet jamais de diagnostic. Les tests du Care Engine vérifient que toute situation ambiguë déclenche une orientation ou une escalade vers le vétérinaire, jamais une conclusion médicale.

1.2 Stratégie de tests par couche

Couche	Outils	Cible
Firmware collier (Rust no_std)	Tests unitaires embarqués + simulateur de capteurs	Logique de collecte/encodage validée hors matériel via le simulateur.
Services backend (Rust)	Tests unitaires (cargo test) + tests d'intégration API	Logique métier et endpoints couverts ; tests d'abus obligatoires sur l'authentification.
Moteur IA / Care Engine (Python)	pytest + jeux d'évaluation RAG + garde-fous anti-diagnostic	Non-régression des réponses et escalade systématique en cas de doute.
App mobile (Kotlin / SwiftUI)	Tests unitaires natifs + tests d'UI critiques	Onboarding, appairage et alertes vérifiés.
Bout en bout (E2E)	Scénarios end-to-end en phase d'intégration	Collier → backend → IA → app/portail validés ensemble.

1.3 Conventions de code

- Linting/formatage automatiques par langage : rustfmt + clippy (Rust), ruff/black (Python), ktlint (Kotlin), ESLint + Prettier (TS/Next).
- Nommage cohérent par langage (snake_case Rust/Python, camelCase Kotlin/TS) et noms descriptifs.
- Toute fonctionnalité passe par une Pull Request relue par au moins un pair avant merge.
- Pas de secret en clair dans le code : variables d’environnement + secrets chiffrés (SOPS).

1.4 Workflow Git

Branche	Rôle
main	Branche stable et déployable. Protégée : merge uniquement via PR validée.

Branche	Rôle
develop	Intégration continue des fonctionnalités avant stabilisation.
feat/<scope>	Une fonctionnalité = une branche, rattachée à un ticket Jira.
fix/<scope>	Correction de bug isolée et traçable.

Commits conventionnels (feat:, fix:, docs:, chore:...) référençant la clé Jira. **Pull Request** : description claire, lien Jira, CI verte et revue d'au moins un pair avant merge.

1.5 CI/CD & déploiement

- À chaque Pull Request : lint + tests automatiques via GitHub Actions (merge bloqué si rouge).
- Build et publication d'images conteneurisées (Docker) versionnées.
- Déploiement GitOps : Helm + Argo CD synchronisent le cluster Kubernetes depuis Git.
- Secrets chiffrés (SOPS + age) ; observabilité OpenTelemetry → Prometheus/Loki/Tempo → Grafana.

1.6 Onboarding développeur

- Accès dépôt GitHub, projet Jira et espace Confluence dès l'arrivée.
- Lecture d'entrée : ce plan qualité, le Git Workflow et l'architecture globale.
- Mise en route locale : clone, installation, lancement en local (README de chaque service).
- Première contribution : un ticket « bonne première tâche », une PR suivant les conventions.